

ABP Innovación y Gestión de Datos

Extracción de Datos (Extract):

Para empezar trabajamos desde Google Colab, abrimos un archivo nuevo y subimos el Dataset previamente descargado del sitio Kaggle, luego cargamos los datos de ventas y clientes desde el archivo CSV en dos Data Frames distintos. Preparamos el entorno importando las librerías que íbamos a utilizar.

- **Importación de bibliotecas**

```
import pandas as pd
import matplotlib.pyplot as plt
```

- **Extracción y creación de ambos Dataframes**

```
url = "/content/customer_data.csv"
```

```
Cientes = pd.read_csv(url, encoding="latin-1") # Usamos
encoding='latin1' para evitar errores al leer archivos con caracteres
especiales (como tildes o eñes)
```

```
url = "/content/sales_data.csv"
```

```
Ventas = pd.read_csv(url, encoding="latin-1")
```

Vista Inicial

- **Comandos empleados para acceder a los dataframes**

Los comandos **.head** y **.info** permiten mostrar la información inicial de ambos dataframes. Esto incluye dimensiones, tipos de datos por columna, y cantidad de valores no nulos, lo cual permite tener una visión general del estado y estructura de los datos antes de cualquier procesamiento.

```
print(Cientes.head())
```

	i»çcustomer_id	gender	age	payment_method
0	C241288	Female	28.0	Credit Card
1	C111565	Male	21.0	Debit Card
2	C266599	Male	20.0	Cash
3	C988172	Female	66.0	Credit Card
4	C189076	Female	53.0	Cash

```
print(Ventas.head())
```

	invoice_no	customer_id	category	quantity	price	invoice_date
0	I138884	C241288	Clothing	5	1500.40	05-08-2022
1	I317333	C111565	Shoes	3	1800.51	12-12-2021
2	I127801	C266599	Clothing	1	300.08	09-11-2021
3	I173702	C988172	Shoes	5	3000.85	16-05-2021
4	I337046	C189076	Books	4	60.60	24-10-2021

	shopping_mall
0	Kanyon
1	Forum Istanbul
2	Metrocity
3	Metropol AVM
4	Kanyon

```
Clientes.info()
```

```
Ventas.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99457 entries, 0 to 99456
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   i»çcustomer_id  99457 non-null  object
1   gender          99457 non-null  object
2   age             99338 non-null  float64
3   payment_method  99457 non-null  object
dtypes: float64(1), object(3)
memory usage: 3.0+ MB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99457 entries, 0 to 99456
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   invoice_no      99457 non-null  object
1   customer_id     99457 non-null  object
2   category        99457 non-null  object
3   quantity        99457 non-null  int64
4   price           99457 non-null  float64
5   invoice_date    99457 non-null  object
6   shopping_mall   99457 non-null  object
dtypes: float64(1), int64(1), object(5)
memory usage: 5.3+ MB
```

Limpieza

Gestión de valores nulos, duplicados, formato de fechas y valores atípicos

```
print("Valores nulos en Clientes:")
print(Clientes.isnull().sum())
```

```
print("\nValores nulos en Ventas:")
print(Ventas.isnull().sum())
```

```
Valores nulos en Clientes:
i»¿customer_id      0
gender              0
age                119
payment_method      0
dtype: int64
```

```
Valores nulos en Ventas:
invoice_no          0
customer_id         0
category            0
quantity            0
price               0
invoice_date        0
shopping_mall       0
dtype: int64
```

```
print("Duplicados en Clientes:")
print(Clientes.duplicated().sum())
```

```
print("\nDuplicados en Ventas:")
print(Ventas.duplicated().sum())
```

```
Duplicados en Clientes:
0
```

```
Duplicados en Ventas:
0
```

```
print("Tipo de dato de 'invoice_date' en Ventas:")
print(Ventas['invoice_date'].dtype)
```

```
Tipo de dato de 'invoice_date' en Ventas:
object
```

```
print("Estadísticas descriptivas de 'age' en Clientes para identificar
posibles atípicos:")
display(Clientes['age'].describe())
```

```
print("\nEstadísticas descriptivas de 'quantity' en Ventas para identificar  
posibles atípicos:")  
display(Ventas['quantity'].describe())
```

```
print("\nEstadísticas descriptivas de 'price' en Ventas para identificar  
posibles atípicos:")  
display(Ventas['price'].describe())
```

count	99338.000000
mean	43.425859
std	14.989400
min	18.000000
25%	30.000000
50%	43.000000
75%	56.000000
max	69.000000

dtype: float64

Estadísticas descriptivas de 'quantity' en Ventas para identificar posibles atípicos:

	quantity
count	99457.000000
mean	3.003429
std	1.413025
min	1.000000
25%	2.000000
50%	3.000000
75%	4.000000
max	5.000000

dtype: float64

Estadísticas descriptivas de 'price' en Ventas para identificar posibles atípicos:

	price
count	99457.000000
mean	689.256321
std	941.184567
min	5.230000
25%	45.450000
50%	203.300000
75%	1200.320000
max	5250.000000

dtype: float64

Realizamos las verificaciones de valores nulos, duplicados, formato de fechas y valores atípicos en los DataFrames Clientes y Ventas.

Encontramos:

- 119 valores nulos en la columna 'age' del DataFrame Clientes.
- No hay valores duplicados en ninguno de los Data Frames.
- La columna 'invoice_date' en el Data Frame Ventas está actualmente como tipo 'object' y la convertimos a formato de fecha.

El siguiente paso fue concatenar los dos Data Frames anteriores, en uno final con toda la información relevante y preparada para un análisis exhaustivo. Renombramos la columna 'i»¿customer_id' en Clientes para que coincida con 'customer_id' en Ventas, realizamos la unión de ambos archivos y obtuvimos un único dataframe "ventas_clientes"

```
Clientes.rename(columns={'i»¿customer_id': 'customer_id'}, inplace=True)
```

```
ventas_clientes = pd.merge(Clientes, Ventas, on='customer_id')
```

```
print("Primeras filas del DataFrame combinado:")  
display(ventas_clientes.head())
```

```
print("\nInformación del DataFrame combinado:")  
ventas_clientes.info()
```

Primeras filas del DataFrame combinado:

	customer_id	gender	age	payment_method	invoice_no	category	quantity	price	invoice_date	shopping_mall
0	C241288	Female	28.0	Credit Card	I138884	Clothing	5	1500.40	2022-08-05	Kanyon
1	C111565	Male	21.0	Debit Card	I317333	Shoes	3	1800.51	2021-12-12	Forum Istanbul
2	C266599	Male	20.0	Cash	I127801	Clothing	1	300.08	2021-11-09	Metrocity
3	C988172	Female	66.0	Credit Card	I173702	Shoes	5	3000.85	2021-05-16	Metropol AVM
4	C189076	Female	53.0	Cash	I337046	Books	4	60.60	2021-10-24	Kanyon

Información del DataFrame combinado:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 99457 entries, 0 to 99456

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	customer_id	99457 non-null	object
1	gender	99457 non-null	object
2	age	99457 non-null	float64
3	payment_method	99457 non-null	object
4	invoice_no	99457 non-null	object
5	category	99457 non-null	object
6	quantity	99457 non-null	int64
7	price	99457 non-null	float64
8	invoice_date	99457 non-null	datetime64[ns]
9	shopping_mall	99457 non-null	object

dtypes: datetime64[ns](1), float64(2), int64(1), object(6)

memory usage: 7.6+ MB

Imputamos los valores faltantes de edad con la mediana

```
mediana_edad = Clientes['age'].median()
```

```
Clientes['age'] = Clientes['age'].fillna(mediana_edad)
```

```
print("Valores nulos en Clientes después de la imputación:")
```

```
print(Clientes.isnull().sum())
```

Valores nulos en Clientes después de la imputación:

```
¿customer_id    0
gender          0
age             0
payment_method  0
dtype: int64
```

Convertimos a formato datetime

```
Ventas['invoice_date'] = pd.to_datetime(Ventas['invoice_date'],
```

```
format='%d-%m-%Y')
```

```
print("Tipo de dato de 'invoice_date' después de la conversión:")
```

```
print(Ventas['invoice_date'].dtype)
```

Tipo de dato de 'invoice_date' después de la conversión:

```
datetime64[ns]
```

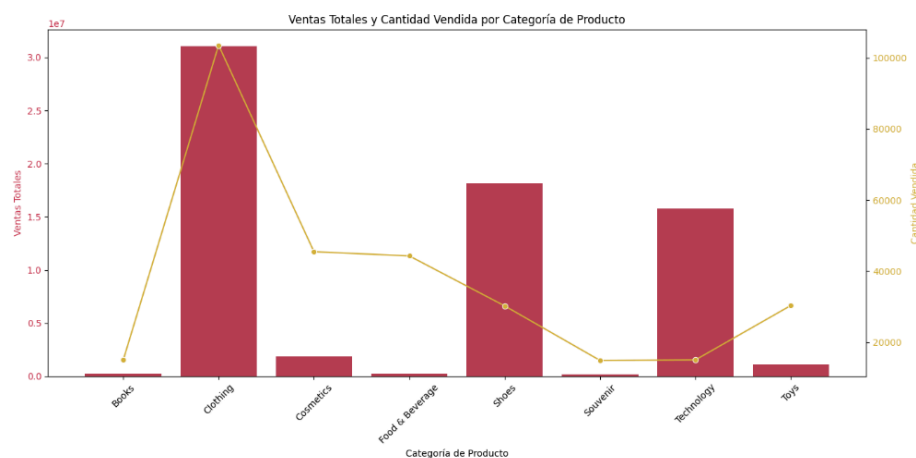
Analizamos y visualizamos los datos del data frame ventas_clientes por categoría de producto, método de pago, género, edad, centro comercial y tiempo.

- **Por categoría de producto**

```
ventas_por_categoria = ventas_clientes.groupby('category')[['price',
'quantity']].sum().reset_index()
print("Ventas y cantidad vendida por categoría:")
display(ventas_por_categoria)
```

Ventas y cantidad vendida por categoría:

	category	price	quantity
0	Books	226977.30	14982
1	Clothing	31075684.64	103558
2	Cosmetics	1848606.90	45465
3	Food & Beverage	231568.71	44277
4	Shoes	18135336.89	30217
5	Souvenir	174436.83	14871
6	Technology	15772050.00	15021
7	Toys	1086704.64	30321

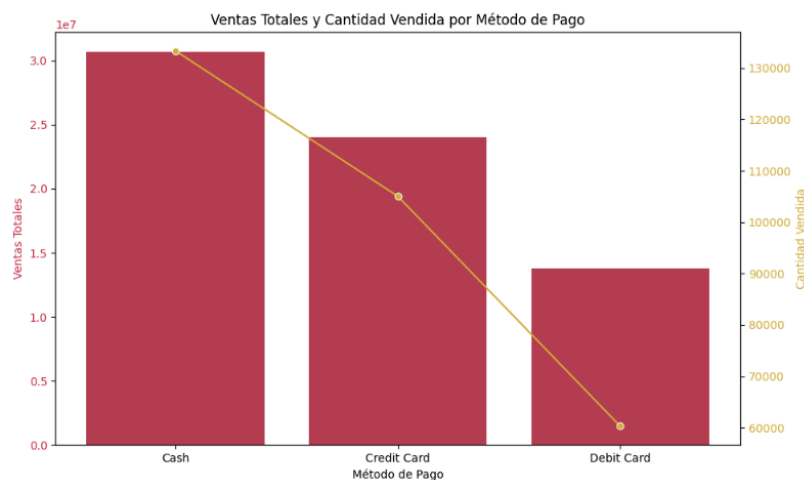


- **Por método de pago**

```
ventas_por_metodo_pago =
ventas_clientes.groupby('payment_method')[['price',
'quantity']].sum().reset_index()
print("Ventas y cantidad vendida por método de pago:")
display(ventas_por_metodo_pago)
```

Ventas y cantidad vendida por método de pago:

	payment_method	price	quantity
0	Cash	30705030.98	133370
1	Credit Card	24051476.93	105045
2	Debit Card	13794858.00	60297



- **Por género**

Agrupamos a los clientes por intervalos de edad, calculamos métricas de ventas por grupo de edad y visualizamos los resultados.

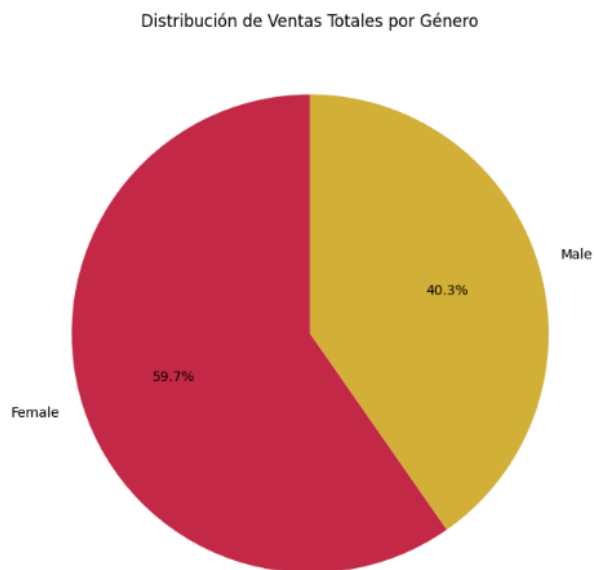
```
ventas_por_genero = ventas_clientes.groupby('gender').agg(
    ventas_totales=('price', 'sum'),
    cantidad_total_vendida=('quantity', 'sum'),
    precio_promedio=('price', 'mean')
).reset_index()

print("Ventas, cantidad vendida y precio promedio por género:")
display(ventas_por_genero)
```

Ventas, cantidad vendida y precio promedio por género:

	gender	ventas_totales	cantidad_total_vendida	precio_promedio
0	Female	40931801.62	178659	688.137615
1	Male	27619564.29	120053	690.920933

Gráfico circular para Ventas Totales por Género



- **Por edad**

Definir intervalos de edad

```
intervalos_edad = [0, 18, 25, 35, 45, 55, 65, 75, 85, 100]
```

```
etiquetas_edad = ['0-17', '18-24', '25-34', '35-44', '45-54', '55-64', '65-74',  
'75-84', '85-100']
```

```
ventas_clientes['intervalo_edad'] = pd.cut(ventas_clientes['age'],  
bins=intervalos_edad, labels=etiquetas_edad, right=False)
```

```
# Calcular métricas de ventas por intervalo de edad
```

```
ventas_por_intervalo_edad =
```

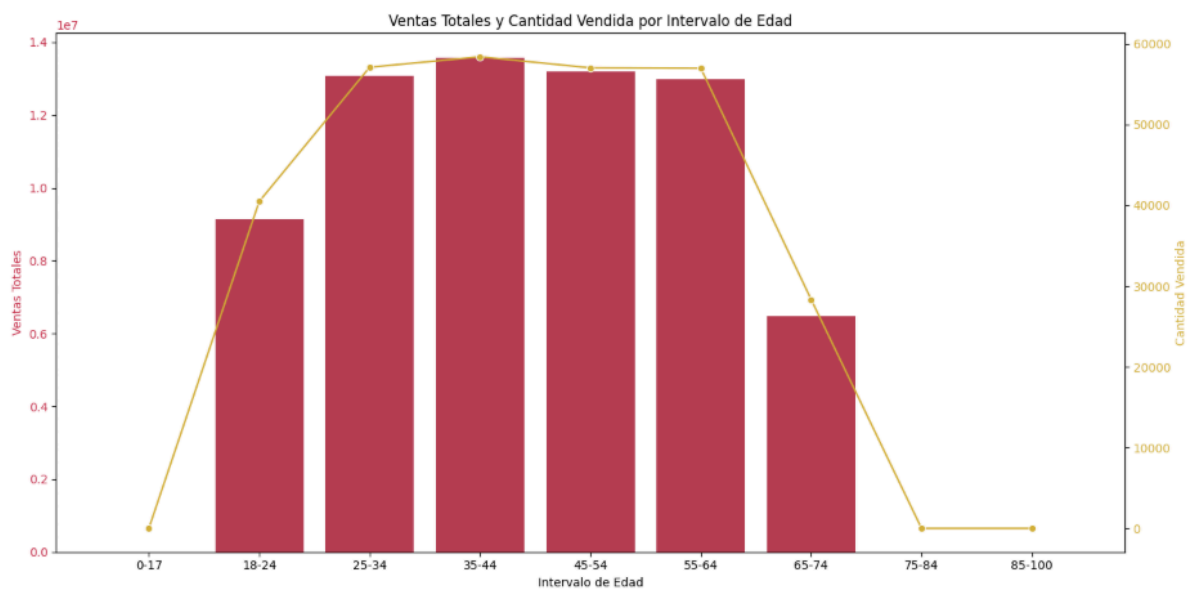
```
ventas_clientes.groupby('intervalo_edad').agg(  
    ventas_totales=('price', 'sum'),  
    cantidad_total_vendida=('quantity', 'sum'),  
    precio_promedio=('price', 'mean')  
)
```

```
    .reset_index()
```

```
print("Ventas, cantidad vendida y precio promedio por intervalo de edad:")
```

```
display(ventas_por_intervalo_edad)
```

	intervalo_edad	ventas_totales	cantidad_total_vendida	precio_promedio
0	0-17	0.00	0	NaN
1	18-24	9153000.17	40487	678.804522
2	25-34	13069076.64	57105	685.932748
3	35-44	13650228.47	58758	700.083520
4	45-54	13195901.20	57032	693.389796
5	55-64	12997078.04	56985	684.741480
6	65-74	6486081.39	28345	689.275387
7	75-84	0.00	0	NaN
8	85-100	0.00	0	NaN



- **Por Centro comercial**

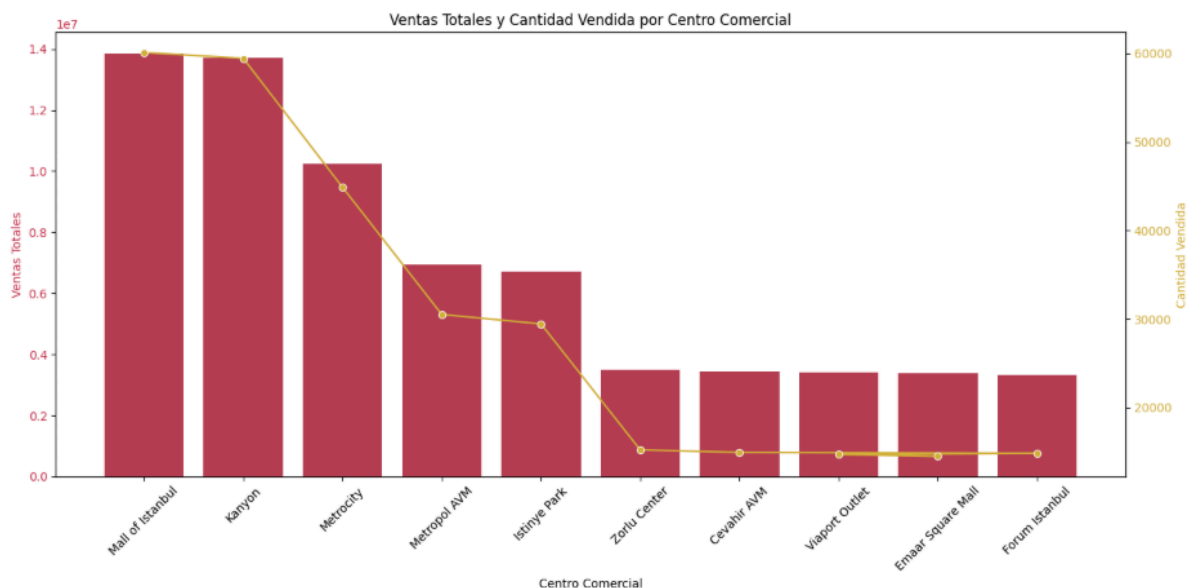
```

ventas_por_centro_comercial =
ventas_clientes.groupby('shopping_mall')[['price',
'quantity']].sum().reset_index()
print("Ventas y cantidad vendida por centro comercial:")
display(ventas_por_centro_comercial)

```

Ventas y cantidad vendida por centro comercial:

	shopping_mall	price	quantity	
0	Cevahir AVM	3433671.84	14949	
1	Emaar Square Mall	3390408.31	14501	
2	Forum Istanbul	3336073.82	14852	
3	Istinye Park	6717077.54	29465	
4	Kanyon	13710755.24	59457	
5	Mall of Istanbul	13851737.62	60114	
6	Metrocity	10249980.07	44894	
7	Metropol AVM	6937992.99	30530	
8	Viaport Outlet	3414019.46	14716	
9	Zorlu Center	3509649.02	15234	



Mapeamos las coordenadas al DataFrame ventas_clientes

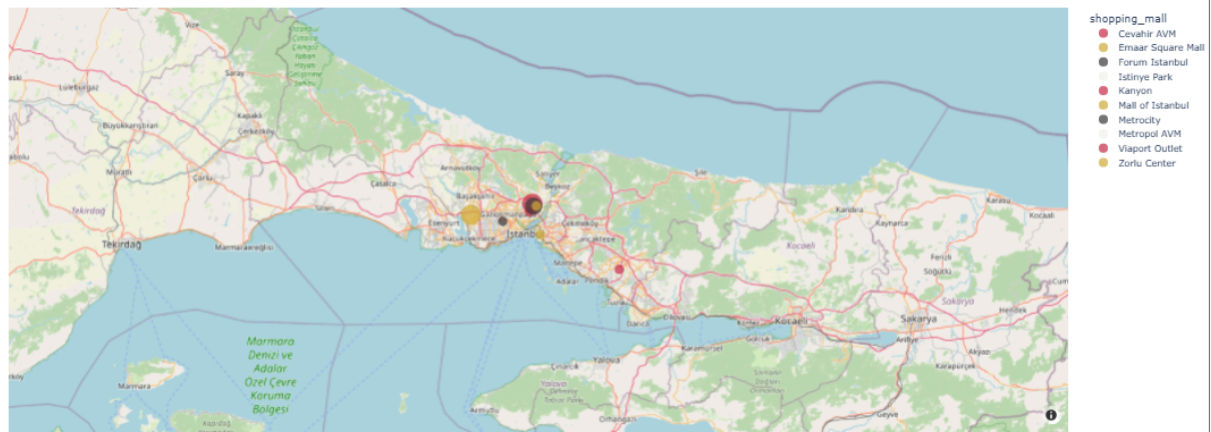
Calculamos las ventas totales por centro comercial para el tamaño de las burbujas

Unimos las coordenadas con los datos de ventas totales por centro comercial

Calculamos el porcentaje de ventas

Creamos el mapa de burbujas interactivo

Distribución de Ventas por Centro Comercial



Porcentaje de ventas por centro comercial:

	shopping_mall	porcentaje_ventas
5	Mall of Istanbul	20%
4	Kanyon	20%
6	Metrocity	15%
7	Metropol AVM	10%
3	Istinye Park	10%
9	Zorlu Center	5%
0	Cevahir AVM	5%
8	Viaport Outlet	5%
1	Emaar Square Mall	5%
2	Forum Istanbul	5%

- **Tendencias en ventas a lo largo del tiempo**

```
ventas_clientes['año'] = ventas_clientes['invoice_date'].dt.year
ventas_clientes['mes'] = ventas_clientes['invoice_date'].dt.month
```

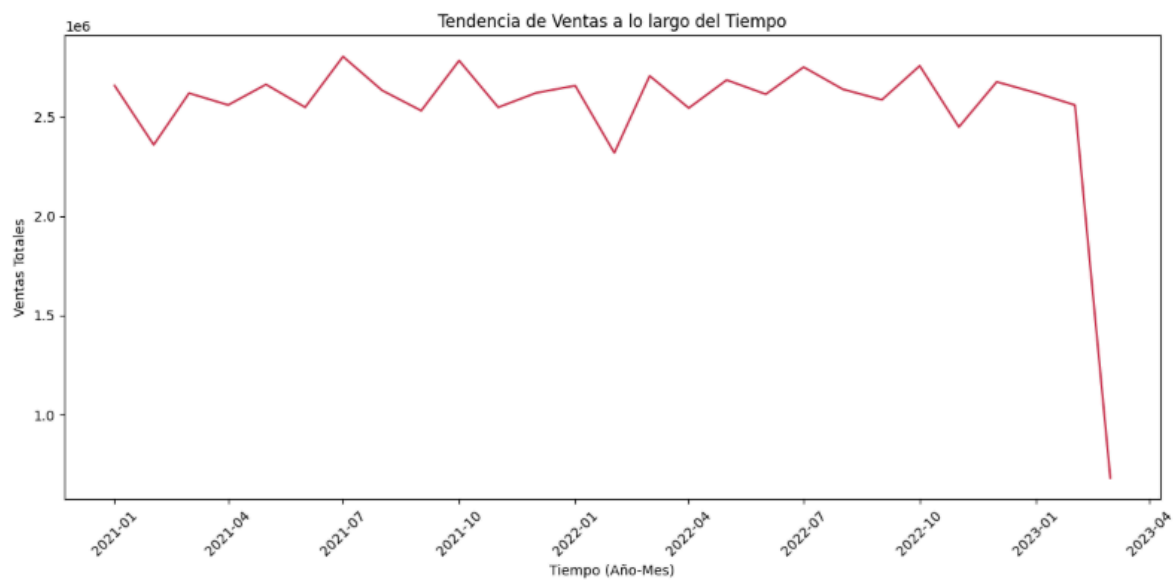
```
tendencia_ventas_a_lo_largo_del_tiempo =
ventas_clientes.groupby(['año', 'mes']).agg(
    ventas_totales=('price', 'sum')
).reset_index()
```

```
tendencia_ventas_a_lo_largo_del_tiempo =
tendencia_ventas_a_lo_largo_del_tiempo.sort_values(by=['año', 'mes'])
```

```
print("Tendencia de ventas a lo largo del tiempo:")
display(tendencia_ventas_a_lo_largo_del_tiempo)
```

Tendencia de ventas a lo largo del tiempo:

	año	mes	ventas_totales	
0	2021	1	2656422.78	
1	2021	2	2358636.34	
2	2021	3	2618434.14	
3	2021	4	2558825.62	
4	2021	5	2662369.93	
5	2021	6	2547239.73	
6	2021	7	2802468.58	
7	2021	8	2632303.32	
8	2021	9	2530305.88	
9	2021	10	2782418.40	
10	2021	11	2547152.35	
11	2021	12	2619727.56	
12	2022	1	2656149.96	
13	2022	2	2318201.08	
14	2022	3	2705190.76	
15	2022	4	2543653.14	
16	2022	5	2684556.89	
17	2022	6	2613106.01	
18	2022	7	2749554.99	
19	2022	8	2638238.71	
20	2022	9	2584908.39	
21	2022	10	2755839.69	
22	2022	11	2447988.76	
23	2022	12	2675437.80	
24	2023	1	2620053.89	
25	2023	2	2558459.90	
26	2023	3	683721.31	



Análisis

KPIs

los KPIs (Indicadores Clave de Rendimiento) que se han analizado son los siguientes:

- **Ventas totales:** La suma de los precios de todas las transacciones.
- **Cantidad total vendida:** La suma de la cantidad de artículos vendidos en todas las transacciones.
- **Número de facturas únicas:** La cantidad de transacciones distintas registradas.
- **Valor promedio por transacción:** El resultado de dividir las ventas totales por el número de facturas únicas.

Calculamos Ventas Totales

```
ventas_totales = ventas_clientes['price'].sum()
```

Cantidad Total Vendida

```
cantidad_total_vendida = ventas_clientes['quantity'].sum()
```

Número de Facturas Únicas (Transacciones)

```
numero_facturas_unicas = ventas_clientes['invoice_no'].nunique()
```

Valor Promedio por Transacción

```
valor_promedio_por_transaccion = ventas_totales /  
numero_facturas_unicas if numero_facturas_unicas > 0 else 0
```

```
print(f"Ventas totales: {ventas_totales:.2f}")  
print(f"Cantidad total vendida: {cantidad_total_vendida}")  
print(f"Número de facturas únicas: {numero_facturas_unicas}")  
print(f"Valor promedio por transacción:  
{valor_promedio_por_transaccion:.2f}")
```

```
Ventas totales: 68551365.91  
Cantidad total vendida: 298712  
Número de facturas únicas: 99457  
Valor promedio por transacción: 689.26
```

Conclusiones

Informe Cliente

https://docs.google.com/document/d/1qbem08EjS1ZdJ2nnpn_p6H0CAEC5ypeRtLRvXb0KxaHA/edit?usp=sharing